KMDAQO: A Knowledge-Managed Drift-Adaptive Query Optimization System

Yiyu Liu
Beijing University of Posts and Telecommunications
Beijing, China
liuyiyu@bupt.edu.cn

Yuanchun Guo
Beijing University of Posts and Telecommunications
Beijing, China
gyc2001@bupt.edu.cn

Bingyan Liu
Beijing University of Posts and Telecommunications
Beijing, China
bingyanliu@bupt.edu.cn

***Abstract*—Workload drift is a major challenge for query optimization in long-running database systems. As query distributions evolve, optimization experience learned from historical workloads may become stale, causing learned and LLM-assisted optimizers to generate suboptimal or harmful decisions for newly emerging queries. Retrieval-augmented optimizers improve adaptability by using historical cases during inference, but their external memory can grow continuously, increasing retrieval cost, prompt length, and exposure to stale examples.**

We propose KMDAQO, a Knowledge-Managed Drift-Adaptive Query Optimizer for LLM-assisted hint generation under workload drift. KMDAQO maintains a bounded optimization knowledge base that stores historical queries, plan-structured features, `pg_hint_plan` hints, execution plans, and observed speedups. For each incoming query, KMDAQO retrieves relevant cases, constructs a structured prompt, generates database hints with a fine-tuned LLM, and validates the output before execution. To handle evolving workloads, KMDAQO monitors execution feedback, triggers offline adaptation when drift is detected, explores candidate hints through replay, and updates the knowledge base with newly observed optimization experience. To avoid unbounded memory growth, high-value knowledge can be selectively internalized into the model through offline knowledge editing, with validation and rollback to prevent harmful updates. Experiments on JOB, JOB-EXT, and CEB under controlled workload drift show that KMDAQO consistently outperforms PostgreSQL, Bao, and retrieval-augmented LLM baselines, reducing average execution latency by 30.1%–72.4%.

***Index Terms*—Query optimization, workload drift, large language models, retrieval-augmented generation, knowledge editing, learned query optimization.**

I. Introduction

Query optimization is a core component of relational database systems, as the quality of execution plans directly affects query latency, resource consumption, and system throughput. Classical optimizers rely on cardinality estimation, cost models, and plan enumeration heuristics to select physical operators, access paths, and join orders [1]–[4]. Adaptive and feedback-driven optimizers have also been explored to refine optimization decisions during or after execution [5]–[7]. Although effective in many scenarios, they remain vulnerable to inaccurate estimates, complex join correlations, and changes in workload distributions. Recent learned optimizers and large language model (LLM) based optimizers provide new opportunities by learning from execution feedback or generating optimization hints from query context [8]–[10].

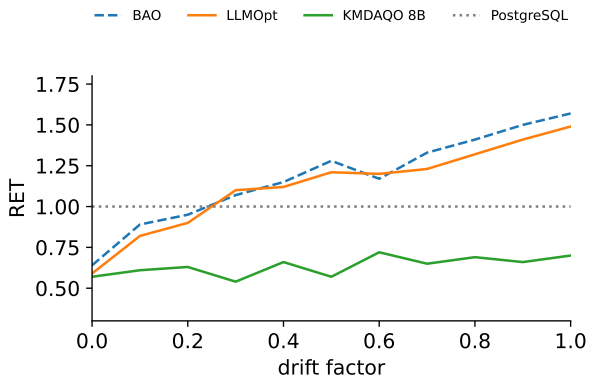

Fig. 1. Optimization performance under workload drift on JOB-derived NeurBench workloads. Each point reports the average RET, defined as $T_{\text{method}}/T_{\text{PostgreSQL}}$, over all queries at the corresponding drift factor. PostgreSQL is the normalized reference line at RET = 1; lower RET is better, and RET > 1 indicates performance regression relative to PostgreSQL. Larger drift factors indicate larger shifts from the original JOB workload in predicate selectivity and query-pattern distribution.

However, these methods still face a critical challenge in long-running database systems: workload drift.

In practical deployments, query workloads evolve over time due to changes in application logic, user behavior, business requirements, and data distributions [11]. As a result, incoming queries may deviate from the workload observed during training, tuning, or previous execution. Under such drift, optimization knowledge that was previously effective may become stale, causing learned and LLM-assisted optimizers to produce suboptimal or even harmful decisions for newly emerging query regions. Figure 1 provides a motivational comparison: as the drift factor increases, BAO [8] and LLMOpt [10] exhibit substantial performance degradation, indicating that learned feedback and one-shot LLM hinting do not generalize reliably to shifted query distributions. LLMOpt is used here to illustrate this basic drift vulnerability of LLM-based optimization, while the main experiments use SEFRQO as the stronger retrieval-augmented LLM baseline that more directly matches KMDAQO's setting. In contrast, KMDAQO 8B maintains lower RET values across most drift levels, indicating stronger robustness under evolving workloads.

Retrieval-augmented LLM optimizers address this issue by retrieving similar historical optimization cases during inference [12]–[14]. These cases may include SQL queries, hints, execution plans, and observed speedups, allowing the model to exploit prior experience when optimizing unseen queries. However, retrieval alone introduces a knowledge management problem. Historical cases are not equally useful: some remain valuable across related queries, while others become stale, redundant, or misleading after workload drift. Without explicit management, an accumulated knowledge base may provide low-quality retrieval results and weaken generalization under evolving workloads.

These observations motivate a deployable LLM-assisted query optimizer with three key capabilities. First, it should adapt when workload drift invalidates previous optimization experience. Second, it should maintain a bounded and high-quality optimization memory instead of simply accumulating all historical records. Third, it should update its knowledge using actual execution feedback while preventing invalid or harmful hints from affecting future optimization.

We propose KMDAQO, a Knowledge-Managed Drift-Adaptive Query Optimizer that treats LLM-assisted query optimization as a closed-loop system rather than a one-shot hint generation task. KMDAQO is built around three technical components. First, a retrieval-guided online hint generation path extracts plan-structured query features, retrieves a small number of relevant historical cases, and uses a fine-tuned LLM to generate `pg_hint_plan` hints with validation and fallback. This component addresses unseen queries while keeping query-time optimization bounded and safe. Second, a drift-aware offline case acquisition path monitors execution feedback to detect degraded optimization behavior, then replays representative drifted queries with candidate hints to collect validated execution outcomes. This component addresses workload drift without forcing every online query to run expensive exploration. Third, a bounded knowledge maintenance path ranks cases by utility, prunes stale or redundant examples, and selectively internalizes stable high-value knowledge through offline knowledge editing with validation and rollback. This component prevents the retrieval memory from growing without control while preserving useful optimization experience for future drifted workloads.

We evaluate KMDAQO on JOB, JOB-EXT, and CEB under controlled workload drift. Compared with PostgreSQL, BAO, and retrieval-augmented LLM baselines, KMDAQO achieves consistently lower execution latency and stronger robustness under increasing drift. Across drifted rounds, KMDAQO reduces average execution latency by 30.1%–72.4%. Ablation studies further show that plan-structured retrieval, drift-aware adaptation, and managed knowledge updating provide complementary benefits.

This paper makes the following contributions:

- **We design KMDAQO**: a closed-loop LLM-assisted query optimization system for long-running workloads under drift.
- **We propose plan-structured retrieval with managed optimization memory**: enabling the system to retrieve, rank, update, and remove historical optimization cases according to their measured value.
- **We introduce drift-aware offline adaptation with safe knowledge updating**: using execution feedback, offline replay, knowledge editing, validation, and rollback to adapt to evolving workloads.
- **We conduct a comprehensive evaluation on multiple benchmarks and drift settings**: showing that KMDAQO improves execution latency, tail robustness, and long-term stability over traditional, learned, and retrieval-augmented baselines.

II. Related Work

A. Hint-Based Query Optimization

Classical query optimizers rely on cardinality estimation, cost models, and search strategies to select access paths, physical operators, and join orders from a large plan space [1]–[4]. Although effective in many settings, they can degrade when estimates are inaccurate or when workloads contain correlations that are not well captured by database statistics. Related work has also studied configuration- or physical-design-aware optimization, where optimizer decisions are evaluated under different system configurations or design choices [15].

Hint-based plan steering provides a lightweight way to influence optimizer behavior without replacing the underlying database optimizer [16]. Hints can constrain access paths, join methods, or join orders, allowing an external model to correct poor plan choices while still relying on the DBMS execution engine. KMDAQO follows this design: it does not rewrite SQL or replace PostgreSQL, but acts as an adaptive hint-generation layer on top of the native optimizer.

B. Learned and LLM-Assisted Query Optimization

Learned query optimization improves or complements cost-based optimization by learning from workload experience, including join ordering, plan selection, and query performance prediction [17]–[21]. Systems such as BAO and Balsa show that learned guidance can improve plan quality while still using existing database infrastructure [8], [22]. Recent work further studies robust plan selection, latency-aware optimization, and learning-to-rank formulations [23]–[32]. A related line learns cardinality estimators for correlated predicates and joins [33]–[35], improves robustness through uncertainty or plan-aware losses [36]–[39], and explores zero-shot or pretrained estimators for broader generalization [40]–[42].

LLM-assisted optimizers use textual SQL input, execution context, and retrieved examples to generate hints, plan choices, or optimization suggestions [9], [10], [13], [14]. These methods are flexible for unseen queries, but they also introduce inference overhead, prompt-construction cost, output-reliability issues, and dependence on historical examples. When the online workload differs from the workload used for training, tuning, or retrieval, previously effective decisions may become

unreliable, making workload drift a central deployment challenge.

C. Workload Drift and Unseen Query Generalization

Real database workloads are rarely stationary. Query templates, predicate values, join patterns, table access frequencies, and selectivity distributions may change as applications, users, or data evolve. Recent benchmark studies show that learned database components can be sensitive to data and workload drift, making robustness under distribution shift important for learned query optimization [11], [43], [44].

For query optimization, drift does not mean that different workload regions should have similar absolute execution times. Instead, it changes the relationship between query features, candidate plans, and effective hints. An optimizer that performs well on one distribution may retrieve irrelevant examples or choose inappropriate hints after the workload shifts. Therefore, we evaluate unseen-query generalization by relative execution performance against PostgreSQL for the same query, rather than by comparing raw latencies across unrelated queries.

D. Retrieval-Augmented Query Optimization

Retrieval-Augmented Generation (RAG) enhances a language model by retrieving relevant external knowledge and incorporating it into the inference context [12]. In query optimization, retrieved knowledge may include historical SQL queries, generated hints, execution plans, cost information, and observed runtime improvements. By conditioning on similar historical cases, retrieval-augmented optimizers can ground decisions in empirical execution evidence rather than relying only on parametric model knowledge [12]–[14], [45].

However, retrieval also introduces systems challenges. If the knowledge base grows without control, retrieval becomes less focused and prompts may contain redundant or stale evidence. Retrieval quality also depends on query representation, since pure SQL-string similarity may miss structurally similar queries. Under workload drift, previously useful cases may become misleading, motivating retrieval and knowledge-management policies that consider plan structure, execution behavior, and long-term memory maintenance.

E. Knowledge Editing for Model Adaptation

Knowledge editing studies how to update specific behaviors or factual associations in a pretrained language model without full retraining. Representative methods such as ROME, MEND, and SERAC modify selected knowledge while attempting to preserve behavior on unrelated inputs [46]–[48]. Recent work extends editing to lifelong settings; for example, WISE uses dual parametric memory and routing to reduce interference across repeated edits [49].

For LLM-assisted query optimization, knowledge editing offers a way to consolidate repeatedly useful associations between query structures, plan patterns, and effective hints. However, the system must decide what knowledge to edit, preserve locality, and account for editing cost. KMDAQO therefore treats knowledge editing as an offline maintenance mechanism with validation and rollback, rather than as part of query-time optimization.

III. Overview

A. Problem Statement

We study LLM-assisted query optimization for long-running analytical workloads whose query distribution evolves over time. Let $\mathcal{Q}_t$ denote the workload distribution at time t, and let $q_t \sim \mathcal{Q}_t$ be an incoming SQL query. The database state, including schema and available indexes, is assumed to be fixed within the evaluated workload, while the query distribution, predicate selectivity, and join patterns may shift over time. Such changes are referred to as workload drift.

For each query q_t, PostgreSQL can generate a default plan without external guidance. KMDAQO instead generates a hint set h_t through an optimizer-guidance function:

$$h_t = O(q_t, K_t, M_t),$$

where K_t is the external optimization knowledge base at time t, and M_t is the current LLM-based hint generator. The generated hint set is injected through `pg_hint_plan` and executed by PostgreSQL. Let $T(q_t, h_t)$ denote the observed execution time of q_t under hint set h_t, and let $T(q_t, h_{\mathrm{pg}})$ denote the execution time of the same query under PostgreSQL's default optimizer, where h_{pg} represents the empty hint. We define the relative execution time as:

$$RET(q_t, h_t) = \frac{T(q_t, h_t)}{T(q_t, h_{\mathrm{pg}})}.$$

A smaller RET indicates better optimization quality, while $RET > 1$ indicates that the generated hint is worse than PostgreSQL for the same query. This per-query normalization avoids comparing the absolute execution times of different queries, which may naturally differ in join structure, selectivity, and computational complexity.

The objective of KMDAQO is to minimize the expected RET over an evolving workload:

$$\min_{O} \ \mathbb{E}_{q_t \sim \mathcal{Q}_t} \left[RET(q_t, O(q_t, K_t, M_t)) \right],$$

while satisfying three system requirements.

First, the optimizer should generalize to unseen queries. An unseen query refers to a query that has not appeared exactly in the knowledge base, but may share structural properties with previously optimized queries, such as similar join graphs, predicate patterns, or plan shapes. The optimizer should therefore exploit reusable optimization experience without relying on exact query matches.

Second, the optimizer should adapt under workload drift. When $\mathcal{Q}_t$ shifts from earlier workload distributions, historical optimization knowledge may become stale or misleading. The system should detect degraded optimization behavior, acquire new execution feedback through offline replay, and update its optimization knowledge for the shifted workload region.

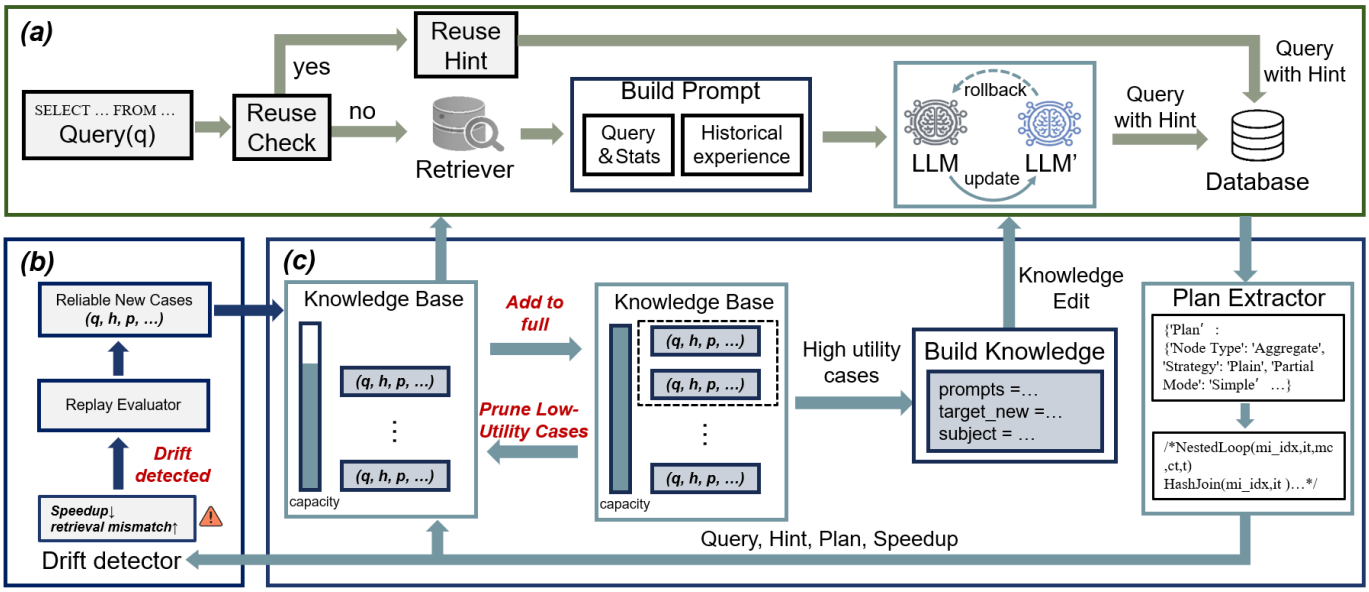

Fig. 2. Overview of KMDAQO. (a) Online Hint Generation Path. (b) Drift-Aware Case Acquisition. (c) Knowledge management and knowledge editing.

Third, the external knowledge base should remain bounded and useful. Let K_t denote the set of stored optimization entries at time t, where each entry contains a query representation, a validated hint set, execution-plan information, observed performance, and failure metadata. KMDAQO maintains:

$$|K_t| \leq B,$$

where B is the capacity of the knowledge base. The purpose of this bound is not only to control storage size, but more importantly to prevent stale, redundant, or harmful cases from dominating retrieval under workload drift. Therefore, the system must decide which entries to retain, update, summarize, internalize, or remove based on their measured optimization value.

In practical deployment, KMDAQO does not require every online query to be executed twice to obtain RET. The online path executes the selected hinted query and records its observed behavior. Baseline comparisons, candidate-hint exploration, and speedup labels are mainly obtained through offline replay or controlled evaluation on selected queries. Thus, execution feedback is used to guide drift adaptation and knowledge updating without requiring a full baseline execution for every live query.

B. KMDAQO System Overview

Figure 2 shows the architecture of KMDAQO, a knowledge-managed drift-adaptive query optimization system. The key idea of KMDAQO is to separate lightweight online hint generation from expensive case acquisition and knowledge maintenance. Specifically, KMDAQO consists of three coordinated paths: an online hint generation path, an offline drift-aware case acquisition path, and a bounded knowledge maintenance path.

As shown in Figure 2(a), the online path handles incoming queries. For each query, KMDAQO first checks whether the query exactly matches a historical case whose stored hint can be reused. If the query cannot be handled by direct reuse, KMDAQO extracts an optimization-oriented representation, such as the query optimization tree and plan features, and retrieves a small number of similar historical cases from the bounded knowledge base. Each case records the query structure, hint, execution plan, and offline-evaluated performance feedback. These cases are organized into a structured prompt and provided to the fine-tuned LLM to generate a hint. The selected hint is then attached to the query and executed by the database. During online execution, KMDAQO only records lightweight feedback, such as the selected hint, latency, and plan information, rather than repeatedly executing each query with multiple candidate hints.

Figure 2(b) illustrates the offline drift-aware case acquisition path. KMDAQO monitors recent online feedback to identify workload changes using signals such as performance degradation, retrieval mismatch, and plan-distribution changes. Once drift is detected, KMDAQO performs offline replay on representative queries from the drifted workload region. In this stage, the system evaluates candidate hints against the default optimizer, collects execution plans and speedups, and constructs reliable optimization cases. These cases provide new experience for adapting to unseen workload patterns without adding expensive evaluation overhead to the online path.

Figure 2(c) shows the bounded knowledge maintenance path. KMDAQO maintains a capacity-limited knowledge base instead of an ever-growing retrieval memory. Newly acquired cases are assigned maintenance utility according to their optimization improvement, recent retrieval access, and redundancy

with existing cases. When the knowledge base reaches its capacity limit, KMDAQO preserves stable high-value knowledge through offline knowledge editing and validates the edited model before deployment. Failed edits are rolled back, while successfully internalized cases are removed from the external knowledge base and no longer participate in top-k retrieval.

Overall, KMDAQO forms a closed-loop system for drift-adaptive query optimization. The online path provides immediate hint generation with bounded query-time operations, the offline path acquires reliable experience under workload drift, and the knowledge maintenance path bounds retrieval cost while preserving useful optimization knowledge.

IV. KMDAQO Design

This section presents the detailed design of KMDAQO. Following the system architecture introduced above, we organize the design around three key mechanisms. First, KMDAQO performs retrieval-guided online hint generation to produce optimization hints for incoming queries. Second, it acquires new optimization knowledge under workload drift by constructing reliable cases from drifted queries. Third, it maintains a bounded knowledge space through utility-based knowledge maintenance, model editing, and pruning. Together, these mechanisms allow KMDAQO to continuously adapt to evolving workloads while keeping the online optimization process lightweight and stable.

A. Retrieval-Guided Online Hint Generation

For each incoming query, KMDAQO follows a lightweight online serving path to generate an optimization hint. The path first checks whether the query has an exact historical match that can be directly reused. If not, KMDAQO retrieves relevant historical cases as contextual evidence and invokes the LLM once to generate a new hint. Expensive adaptation operations, such as candidate hint exploration and knowledge editing, are not performed in the online path.

Given an incoming query q, KMDAQO normalizes it into a canonical form $\kappa(q)$ by removing formatting differences and standardizing syntactic variants that do not change query semantics. The reuse path is triggered only when the canonicalized query exactly matches a historical query in the knowledge base. Similar but non-identical queries are not directly assigned historical hints; instead, they are handled by retrieval-guided LLM generation.

Formally, KMDAQO selects the reuse path when

$$\textsc{Reuse}(q) = \begin{cases} \texttt{True}, & \exists c^* \in \mathcal{K} \text{ s.t. } \kappa(c^*.q) = \kappa(q) \\ & \quad \wedge\ T(c^*) \ll T_{LLM}, \\ \texttt{False}, & \text{otherwise.} \end{cases}$$

Here, $c^*.q$ denotes the historical query stored in case c^*, $T(c^*)$ denotes the recorded hinted execution time of this historical query, and T_{LLM} denotes the estimated latency of one LLM inference. When the condition holds, KMDAQO directly reuses the hint stored in c^*. This avoids unnecessary LLM calls for repeated short queries, where regenerating a hint would introduce inference overhead that is disproportionate to the query execution time.

If the reuse condition is not satisfied, KMDAQO follows the retrieval-guided LLM generation path. It first builds a query representation $\phi(q)$ that captures both the logical query structure and optimizer-derived plan features, such as involved relations, join predicates, estimated operators, and estimated cardinalities. We encode $\phi(q)$ as a fixed-dimensional vector and use cosine similarity to compare the incoming query with historical cases. KMDAQO then retrieves the top-k most relevant cases from the knowledge base:

$$\mathcal{R}(q) = \mathrm{TopK}_{c \in \mathcal{K}} \cos(\phi(q), \phi(c.q)).$$

Each retrieved case contains a historical query, its hint, execution plan, execution time, and observed optimization effect. Since k is fixed during the experiments and kept unchanged in the sensitivity studies, the amount of retrieved evidence and the resulting prompt size remain bounded during online serving.

KMDAQO then constructs a structured prompt for the LLM. The prompt contains the target query, database schema information, the retrieved cases, and an instruction that restricts the output to pg_hint_plan-compatible hints. The retrieved cases provide concrete examples of how related queries were optimized, but their hints are not directly applied unless the exact-query reuse condition holds. The LLM uses this context to generate a hint for the current query, such as a join order, join method, or scan method hint.

The selected hint, either reused from history or generated by the LLM, is checked before execution. If the hint is syntactically valid and applicable to the target query, KMDAQO attaches it to the query and submits the hinted query to PostgreSQL. Otherwise, the system falls back to the original PostgreSQL optimizer for safe execution. During execution, KMDAQO records lightweight runtime information, including the query representation, selected hint, selected plan, execution time, and retrieval confidence. These records are used by later drift-aware case acquisition, but they do not trigger online candidate exploration.

Algorithm 1 summarizes the online hint-generation path of KMDAQO. Overall, retrieval-guided online hint generation allows KMDAQO to exploit accumulated optimization knowledge while keeping the query-time procedure bounded. The online stage either reuses a stored hint for an exact repeated short query or performs one retrieval-augmented LLM invocation to generate a new hint.

B. Drift-Aware Case Acquisition

While the online path uses existing knowledge to optimize incoming queries, KMDAQO must also acquire new cases when the workload distribution changes. To this end, KMDAQO maintains a drift-aware case acquisition mechanism that converts recently observed queries into reliable optimization cases. This mechanism is separated from online hint generation: the online path only records lightweight execution information, while candidate exploration and case validation are performed in a subsequent adaptation phase.

Algorithm 1: Retrieval-Guided Online Hint Generation

Input: Incoming SQL query q; bounded knowledge base $\mathcal{K}$; canonicalizer $\kappa(\cdot)$; query representation function $\phi(\cdot)$; retriever $\mathcal{R}$; top-k parameter k; LLM optimizer M; estimated LLM inference latency T_{LLM}

Output: Selected hint h; execution plan p; execution time T; online execution log ℓ

// Step 1: Canonicalize the incoming query;
$\bar{q} \leftarrow \kappa(q)$;
$c^* \leftarrow \textsc{FindExactMatch}(\bar{q}, \mathcal{K})$;
// Step 2: Fast reuse path for exact repeated short queries;
if $c^* \neq \varnothing$ ***and*** $\kappa(c^*.q) = \bar{q}$ ***and*** $T(c^*) \ll T_{\mathrm{LLM}}$ **then**
  $h \leftarrow c^*.h$;
  $\rho \leftarrow 1.0$;
  path $\leftarrow$ `reuse`;
else
  // Step 3: Retrieval-guided LLM generation path;
  $z_q \leftarrow \phi(q)$;
  $C \leftarrow \mathrm{TopK}_{c \in \mathcal{K}}\, \mathrm{sim}(z_q, \phi(c.q), k)$;
  $\rho \leftarrow \textsc{RetrievalConfidence}(z_q, C)$;
  // Step 4: Build structured prompt and invoke LLM once;
  $S \leftarrow \textsc{FormatRetrievedCases}(C)$;
  $P \leftarrow \textsc{BuildPrompt}(q, S)$;
  $h \leftarrow M(P)$;
  path $\leftarrow$ `llm`;
// Step 5: Validate generated or reused hint;
if ***not*** $\textsc{ValidHint}(h, q)$ **then**
  $h \leftarrow \varnothing$;
  path $\leftarrow$ `fallback`;
// Step 6: Execute query and record lightweight runtime evidence;
$(p, T) \leftarrow \textsc{ExecuteWithPostgreSQL}(q, h)$;
$\ell \leftarrow (q, \phi(q), h, p, T, \rho, \mathit{path})$;
$\textsc{AppendOnlineLog}(\ell)$;
return h, p, T, ℓ;

During online serving, KMDAQO records an execution log for each processed query:

$$\ell_i = (q_i, h_i, p_i, T_i, \rho_i),$$

where q_i is the query, h_i is the selected hint, p_i is the resulting execution plan, T_i is the observed execution time, and ρ_i denotes the retrieval confidence. These records provide runtime evidence about whether the current workload remains close to previously observed cases. KMDAQO monitors recent logs using a sliding window $\mathcal{W}_t$ and computes a drift score:

$$D_t = \frac{1}{|\mathcal{W}_t|} \sum_{\ell_i \in \mathcal{W}_t} d(\ell_i),$$

where $d(\ell_i)$ measures the mismatch between the current query behavior and historical knowledge. In our implementation, it combines three normalized signals: the drop in retrieval confidence ρ_i, the latency deviation from similar historical cases, and a binary hint-failure indicator that records invalid or inapplicable generated hints. When the averaged score D_t exceeds a threshold calibrated on the warm-up workload, KMDAQO marks the recent window as a candidate drift region.

For queries in the drift region, KMDAQO performs replay-based case acquisition. Given a query q_i, the system constructs a candidate hint set $\mathcal{H}(q_i)$ from multiple sources, including the online generated hint, hints retrieved from related historical cases, and common pg_hint_plan-compatible hints. These candidates are evaluated in the adaptation phase to identify the most effective optimization decision:

$$h_i^* = \arg \min_{h \in \mathcal{H}(q_i)} T(q_i, h),$$

where $T(q_i, h)$ denotes the measured execution time of query q_i under hint h during replay. KMDAQO also obtains the default PostgreSQL execution time $T(q_i, \emptyset)$ in this phase, which is used to estimate the optimization effect of the selected hint.

A new case is considered reliable only when the selected hint is valid and provides a measurable improvement over the default optimizer. Formally, KMDAQO constructs a new case

$$c_i = (q_i, h_i^*, p_i^*, T(q_i, h_i^*), s_i),$$

where p_i^* is the execution plan produced under h_i^*, and

$$s_i = \frac{T(q_i, \emptyset)}{T(q_i, h_i^*)}$$

denotes the observed speedup. Cases with unstable execution behavior, invalid hints, or negligible improvement are discarded. The remaining reliable cases form $\mathcal{D}_{new}$, which is then passed to the bounded knowledge maintenance mechanism.

This acquisition process allows KMDAQO to update its optimization knowledge according to newly observed workload patterns. Instead of treating every online query as an opportunity for trial-and-error optimization, KMDAQO uses online observations to identify where new knowledge is needed and then constructs validated cases through replay-based evaluation. As a result, the knowledge base evolves with the workload while the query-time serving procedure remains lightweight.

C. Bounded Knowledge Maintenance

The drift-aware acquisition process continuously produces new reliable cases as the workload evolves. If all historical cases were kept in the external knowledge base, retrieval would gradually become less focused and the prompt context

would contain redundant or outdated evidence. KMDAQO therefore maintains a bounded knowledge space that keeps useful cases externally retrievable while selectively internalizing stable knowledge into the LLM.

Each case in the knowledge base is represented as

$$c = (q, h, p, T, s),$$

where q is the query, h is the selected hint, p is the corresponding execution plan, T is the hinted execution time, and s is the observed speedup over the default optimizer. When a set of newly acquired cases $\mathcal{D}_{new}$ arrives, KMDAQO inserts them into the knowledge base $\mathcal{K}$ and assigns each case a maintenance utility. The utility estimates how valuable a case is for future online hint generation:

$$U(c) = I(c) + A(c) - R(c),$$

where $I(c)$ measures the optimization improvement brought by the stored hint, $A(c)$ measures the recent access frequency of the case during retrieval, and $R(c)$ measures its redundancy with other cases in the knowledge base. In our implementation, $I(c)$ is derived from the observed speedup s, $A(c)$ is updated according to how often the case is retrieved in recent online windows, and $R(c)$ is computed as the maximum representation similarity between c and the remaining cases:

$$R(c) = \max_{c' \in \mathcal{K}, c' \neq c} \mathrm{sim}(\phi(c.q), \phi(c'.q)).$$

Before computing $U(c)$, the three signals are normalized within the current knowledge base, so the utility acts as a deterministic ranking rule that favors cases with clear measured improvement and recent reuse while penalizing near-duplicate cases. The capacity B is selected according to the external-memory and prompt-budget constraint, and its sensitivity is evaluated by halving and doubling the default capacity in Section V-E. Cases with high utility are more likely to be retained or internalized, while cases with low improvement, low access frequency, or high redundancy are candidates for pruning.

When $|\mathcal{K}| > B$, KMDAQO first selects a subset of stable and high-utility cases $\mathcal{K}_{edit}$ for knowledge editing. The goal of this step is not to memorize the entire case verbatim, but to internalize the optimization decision encoded by the case. Specifically, each editing example is constructed as a mapping from an optimization-aware query context to the desired hint decision:

$$e = (x_c, y_c),$$

where x_c contains the query structure, relevant schema context, and plan features of case c, and y_c is the corresponding hint decision, such as a join order, join method, or scan method hint. Through knowledge editing, KMDAQO encourages the LLM to produce the same type of hint when it later observes a query context with a similar optimization pattern.

Given the editing examples, KMDAQO applies WISE-based knowledge editing to obtain a candidate model $\mathcal{M}'$. The edited model is accepted only after validation on a held-out validation set $\mathcal{V}$. This validation checks whether the edited model can reproduce the intended hint decisions while preserving its behavior on existing cases. If the validation succeeds, KMDAQO replaces $\mathcal{M}$ with $\mathcal{M}'$ and removes $\mathcal{K}_{edit}$ from the external knowledge base. After removal, these edited cases no longer participate in top-k retrieval or appear in the prompt. Their effect is instead expected to be reflected through the edited model parameters. Other non-edited cases, including cases that are similar but complementary, remain in $\mathcal{K}$ and can still be retrieved for future queries. If validation fails, KMDAQO discards $\mathcal{M}'$ and keeps both the original model and the external knowledge base unchanged.

After the editing step, KMDAQO enforces the capacity bound on the external knowledge base. If $|\mathcal{K}|$ still exceeds B, the system iteratively removes the case with the lowest utility until the capacity constraint is satisfied:

$$c_{min} = \arg\min_{c \in \mathcal{K}} U(c), \qquad \mathcal{K} \leftarrow \mathcal{K} \setminus \{c_{min}\}.$$

This pruning step removes cases that are less useful for future hint generation, especially cases that provide limited speedup, are rarely retrieved, or are largely covered by other cases.

Algorithm 2 summarizes the bounded knowledge maintenance procedure. By combining improvement-aware utility estimation, knowledge editing, validation, and pruning, KMDAQO prevents the external knowledge base from growing without bound while preserving useful optimization knowledge for future queries.

Algorithm 2: Bounded Knowledge Maintenance

Input: New reliable cases $\mathcal{D}_{new}$, knowledge base $\mathcal{K}$, LLM $\mathcal{M}$, capacity limit B, validation set $\mathcal{V}$, and utility function $U(\cdot)$.
Output: Updated knowledge base $\mathcal{K}$ and LLM $\mathcal{M}$.

$\mathcal{K} \leftarrow \mathcal{K} \cup \mathcal{D}_{new}$;
foreach $c \in \mathcal{K}$ **do**
  $c.u \leftarrow U(c)$;
if $|\mathcal{K}| \leq B$ **then**
  return $\mathcal{K}, \mathcal{M}$;
$\mathcal{K}_{edit} \leftarrow$ SELECTINTERNALIZABLE($\mathcal{K}$);
if $\mathcal{K}_{edit} \neq \emptyset$ **then**
  $\mathcal{E} \leftarrow$ BUILDEDITEXAMPLES($\mathcal{K}_{edit}$);
  $\mathcal{M}' \leftarrow$ KNOWLEDGEEDIT($\mathcal{M}, \mathcal{E}$);
  $b \leftarrow$ VALIDATE($\mathcal{M}', \mathcal{V}$);
  if $b =$ `True` **then**
    $\mathcal{M} \leftarrow \mathcal{M}'$;
    $\mathcal{K} \leftarrow \mathcal{K} \setminus \mathcal{K}_{edit}$;
  else
    DISCARD($\mathcal{M}'$);
foreach $c \in \mathcal{K}$ **do**
  $c.u \leftarrow U(c)$;
while $|\mathcal{K}| > B$ **do**
  $c_{min} \leftarrow \arg\min_{c \in \mathcal{K}} c.u$;
  $\mathcal{K} \leftarrow \mathcal{K} \setminus \{c_{min}\}$;
return $\mathcal{K}, \mathcal{M}$;

V. EVALUATION

We evaluate KMDAQO from two perspectives: its generalization ability under drifting workloads and its optimization capability for complex queries.

TABLE I
MAPPING BETWEEN EVALUATION ROUNDS AND DRIFT FACTORS.

Round	1,5	2,6	3,7	4,8
Drift Factor	0.00	0.33	0.66	1.00

A. Experiment Setup

Benchmarks. We evaluate KMDAQO on three benchmarks with different schema complexity and workload characteristics. **JOB** [4] is a widely used analytical workload built on the IMDB schema. **JOB-EXT** [22] extends JOB with additional query variants and broader query patterns, providing a more challenging setting for evaluating generalization. **CEB** [50] is a cross-domain execution benchmark with template-based queries that exhibit diverse query structures. To evaluate robustness under workload drift, we use **NeurBench** [11] to construct drifted query sets for each benchmark. The drift factor controls the degree of distribution shift in the generated workload, allowing us to evaluate optimizers under progressively stronger workload changes.

Drift Protocol. For each benchmark, we use NeurBench to generate drifted workloads with four drift factors: 0.00, 0.33, 0.66, and 1.00. In NeurBench, the drift factor controls the strength of the distribution shift applied during workload generation. A value of 0.00 produces a workload close to the original benchmark distribution, while larger values generate queries with stronger changes in predicate selectivity, accessed query regions, and query-pattern distribution. Therefore, the four drift factors provide progressively harder test workloads for evaluating whether an optimizer can handle shifted query distributions.

For each drift factor, NeurBench produces a corresponding drifted query set, and all optimizers are evaluated on the same generated queries. To study both initial adaptation and knowledge retention, we organize the evaluation into eight rounds. Rounds 1–4 correspond to the first pass over the four NeurBench-generated drifted workloads, while rounds 5–8 revisit the same drift factors to test whether the optimizer can reuse or preserve optimization knowledge acquired from earlier drifted workloads. Table I summarizes the mapping between evaluation rounds and drift factors.

Baselines. We compare KMDAQO with three representative baselines. **PostgreSQL** is used as the classical cost-based optimizer baseline and provides a non-neural reference. **BAO** [8] represents a learned query optimization method that uses experience from previous executions to guide plan selection. **SEFRQO** [13] represents a retrieval-augmented LLM-based query optimization method. BAO and SEFRQO are evaluated using their default settings. All baselines are evaluated under the same workloads, hardware environment, and execution-time metrics. Other learned hinters, ranking-based optimizers, and hybrid methods such as FASTgres, Lero, and AutoSteer are discussed as related learned optimizers, while the experiments focus on these three baselines to cover native, learned-feedback, and retrieval-augmented LLM optimization.

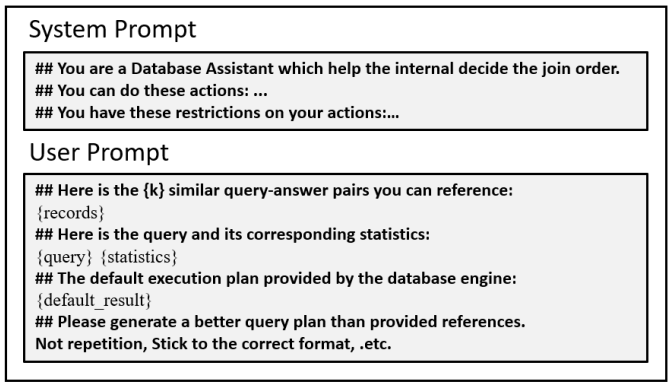

Fig. 3. Unified prompt template used for LLM-based optimizers.

All methods receive the same initial workload, online query stream, execution feedback budget, and replay budget where applicable.

Models and Prompting. We evaluate two KMDAQO model variants, **LSG3B** and **LSG8B**, which are derived from LLaMA foundation models with 3B and 8B parameters, respectively [9]. Both models are fine-tuned through supervised fine-tuning (SFT) and quality-guided prompt refinement (QG-PRO). During inference, we adopt the prompt template used by SEFRQO as the base input format to keep the prompting protocol consistent across LLM-based methods. The template contains the SQL query, relevant query or plan statistics, and retrieved contextual examples when retrieval is enabled. We keep the retrieval top-k fixed across all main experiments and sensitivity studies. Figure 3 shows the prompt layout.

Hardware and Software Configuration. All experiments are conducted on a server equipped with an NVIDIA RTX 4090 GPU. The software environment uses CUDA 11.8 and the same dependency configuration for all evaluated methods. We fix random seeds and use consistent execution settings across models and baselines to reduce variation unrelated to the optimization method.

Metrics. We use Relative Execution Time (RET) as the primary metric. For a query, let $T_{\text{optimized}}$ denote the execution time obtained by the evaluated optimizer and T_{pg} denote the execution time obtained by PostgreSQL. RET is defined as:

$$\text{RET} = \frac{T_{\text{optimized}}}{T_{\text{pg}}}.$$

A smaller RET indicates better performance relative to PostgreSQL. In particular, RET < 1 means the optimizer improves over PostgreSQL, while RET > 1 indicates that the generated optimization decision slows down execution.

B. Overall Performance under Workload Drift

We first evaluate the overall optimization performance of KMDAQO under workload drift. Figures 4, 5, and 6 report the average database execution latency on JOB, JOB-EXT, and CEB, respectively. Table I shows the mapping between evaluation rounds and drift factors. Rounds 1–4 measure the initial response of each optimizer to increasingly shifted

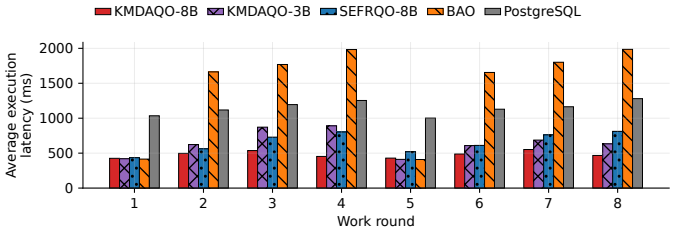

Fig. 4. Execution latencies on JOB.

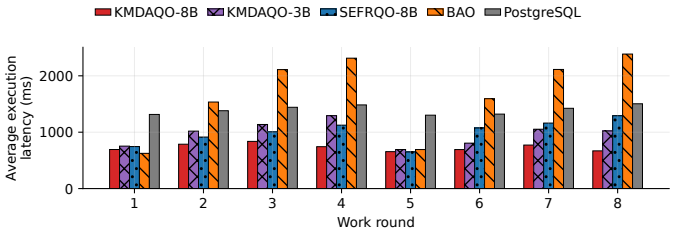

Fig. 5. Execution latencies on JOB-EXT.

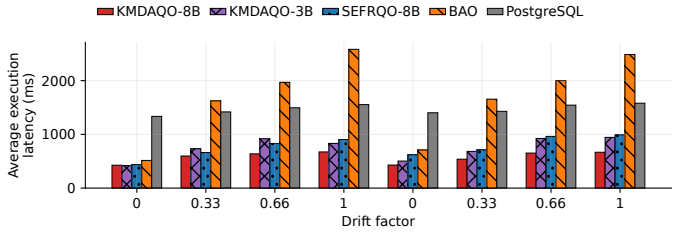

Fig. 6. Execution latencies on CEB.

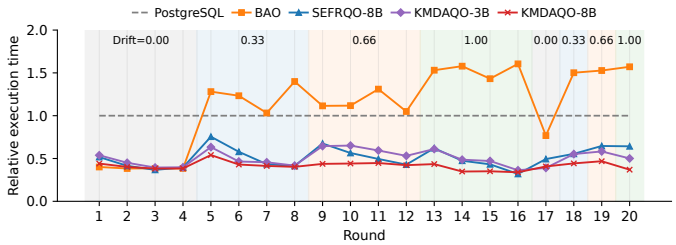

Fig. 7. Query-level RET trajectory under sequential workload drift. The background shading indicates the drift factor of each query segment, and the dashed line at RET=1 denotes the PostgreSQL baseline.

workloads, while Rounds 5–8 repeat the same drift levels to evaluate whether the optimizer can retain and reuse optimization experience acquired from earlier rounds. Across drifted rounds, KMDAQO-8B reduces average execution latency by 58.1

PostgreSQL exhibits relatively stable but consistently higher latency than KMDAQO. Since PostgreSQL does not learn from historical workload feedback, its latency change under drift should not be interpreted as an adaptation failure. Instead, the increase mainly reflects the greater difficulty of the generated drift workloads, such as changed predicate distributions, selectivities, and query structures. PostgreSQL still relies on its native cost model and database statistics, and therefore provides a strong non-neural reference point. However, its cost-based decisions may still suffer from cardinality estimation errors and suboptimal join choices, which limits its performance on difficult drifted queries.

BAO shows stronger degradation as the drift factor increases. Although BAO can learn useful optimization behavior from prior execution feedback, its learned policy is sensitive to the workload distribution from which the feedback is collected. When the workload shifts, previously effective hinting decisions may no longer transfer to the new query region. As a result, BAO may require additional feedback or retraining before it can adapt, leading to delayed response under sudden workload drift. In high-drift rounds, this delay can even result in negative optimization effects, where the selected plan is slower than PostgreSQL's default plan.

SEFRQO performs better than BAO in several low-drift settings because retrieval-augmented prompting allows the LLM to reuse historical optimization examples. However, its advantage becomes less stable as the drift factor increases. Under substantial workload shift, the retrieved historical cases may become less representative of the current query distribution. In addition, because SEFRQO mainly relies on accumulating external retrieval examples, it may require a larger number of relevant cases before it can provide high-quality guidance for newly emerging workload regions. This makes its adaptation slower than KMDAQO when the workload changes rapidly.

In contrast, KMDAQO maintains more robust performance across both initial drift exposure and repeated drift rounds. This improvement comes from the combination of online retrieval-guided hint generation, offline drift adaptation, and bounded knowledge management. Retrieval provides immediate access to similar historical optimization experience, while drift adaptation introduces new execution feedback for emerging workload regions. Bounded knowledge management further prevents the retrieval context from being dominated by stale or redundant cases during long-running deployment. As a result, KMDAQO can preserve useful historical experience while still adapting to newly shifted queries.

Figure 7 further shows the query-level RET trajectory under a sequential drift stream. Unlike Figures 4, 5, and 6, which report average performance over evaluation rounds, this figure illustrates how each optimizer reacts immediately after the workload enters a new drift region. The results show that KMDAQO adapts more quickly when workload drift occurs: its RET may increase slightly at a drift boundary, but it quickly returns to a stable range in the following queries. In contrast, BAO shows much larger degradation after drift, especially under higher drift factors, where its RET remains above the PostgreSQL baseline for multiple consecutive queries. SEFRQO-8B benefits from retrieval-augmented prompting, but its recovery is less stable than KMDAQO, suggesting that retrieval alone is insufficient when the retrieved cases are not well aligned with the shifted workload.

The final four queries revisit drift factors that have appeared earlier in the sequence, and KMDAQO still maintains low

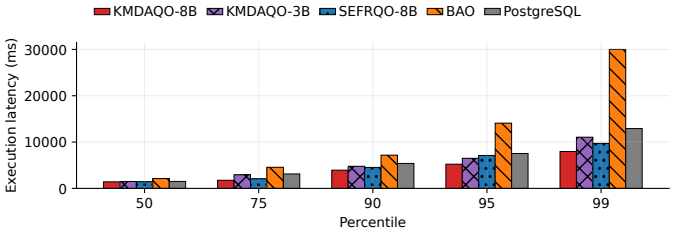

Fig. 8. The 50th, 75th, 90th, 95th, and 99th percentile execution latencies of Execution Latency in PostgreSQL, BAO, KMDAQO-8B, KMDAQO-3B and SEFRQO-8B.

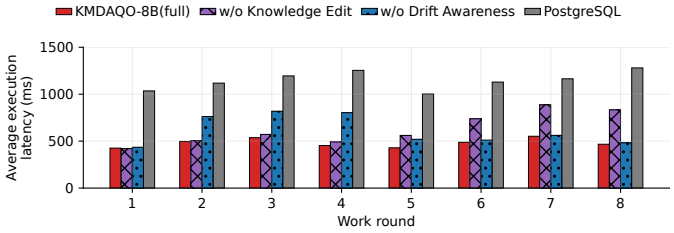

Fig. 9. Component Ablation on JOB.

RET in this stage. This indicates that the system can retain and reuse optimization experience acquired from previous drifted regions, supporting the design of bounded knowledge management. The comparison between KMDAQO-3B and KMDAQO-8B further shows the impact of model capacity: KMDAQO-8B is generally more stable under high drift, while KMDAQO-3B still benefits from the framework but is less robust on complex or heavily drifted workloads.

C. Tail Query Latency Evaluation

Average execution latency does not fully reflect optimizer robustness under workload drift, because a small number of severely suboptimal plans can dominate the total execution time in analytical workloads. We therefore further evaluate tail query latency on JOB-EXT, which contains more diverse query variants and exposes the optimizers to more challenging shifted query patterns. Figure 8 reports the 50th, 75th, 90th, 95th, and 99th percentile database execution latencies of PostgreSQL, BAO, SEFRQO-8B, KMDAQO-3B, and KMDAQO-8B.

The results show that KMDAQO-8B consistently achieves the lowest latency across the evaluated percentiles. This indicates that KMDAQO does not merely improve the average execution time, but also reduces the probability of generating extremely slow plans under workload drift. The advantage becomes more pronounced at higher percentiles, where query optimization errors are more costly. At the 99th percentile, KMDAQO-8B still maintains a latency of approximately 7957 ms. This represents a substantial reduction in tail latency compared with the baselines: 38.4% over PostgreSQL, 73.5% over BAO, and 18.3% over SEFRQO-8B. These results show that KMDAQO not only improves average latency but also avoids the catastrophic slow plans that dominate high-percentile performance under workload drift.

PostgreSQL provides a relatively stable baseline, but its tail latency remains higher than KMDAQO-8B. This is because the native cost-based optimizer may still select inefficient join orders or join methods when cardinality estimates are inaccurate, especially for complex multi-join queries in JOB-EXT. Since PostgreSQL does not use historical execution feedback, it cannot adapt its plan choices to recurring failure patterns observed under drifted workloads.

BAO exhibits the most severe tail degradation. While BAO can improve plan selection when the workload is close to previously observed feedback, its learned policy becomes less reliable under drift. At higher percentiles, BAO reaches the execution-time cap of 30,000 ms, indicating that it can produce catastrophic plans for difficult drifted queries. This result suggests that delayed adaptation is particularly harmful for tail latency: even if an optimizer performs reasonably on many queries, a few poorly optimized queries can dominate high-percentile latency.

SEFRQO-8B performs better than BAO at most percentiles, demonstrating the benefit of retrieval-augmented LLM optimization. However, its tail latency remains higher than KMDAQO-8B. This gap suggests that retrieval alone is insufficient when the workload has shifted substantially and the retrieved historical cases are not fully aligned with the current query region. Without explicit drift-aware acquisition and bounded knowledge management, the optimizer may still rely on stale or incomplete evidence when generating hints for difficult tail queries.

KMDAQO-3B shows a different trade-off. It achieves competitive latency at lower and middle percentiles, indicating that smaller LLMs can benefit from the same knowledge-managed optimization framework. However, its performance becomes less stable at higher percentiles, especially at the 95th and 99th percentiles. This suggests that model capacity matters more for complex tail queries, where the optimizer must interpret multiple retrieved cases, reason about join interactions, and generate coordinated hints. Nevertheless, KMDAQO-3B remains useful as a lower-cost variant when inference resources are constrained.

D. Ablation Study

In this subsection, we analyze the impact of two crucial components of KMDAQO: knowledge management and drift awareness, through an ablation study. The results, as shown in Figure 9, demonstrate the effects of removing these components during the initial adaptation to workload drift and subsequent rounds.

When knowledge management and knowledge editing are removed, the optimizer performs similarly to the full KMDAQO in the initial rounds (Rounds 2-4), where the impact of workload drift is not significantly pronounced. However, in rounds 6-8, as the accumulation of knowledge increases, the efficiency of the RAG system decreases. This results in a reduced ability to provide high-quality experiences to assist the LLM inference, leading to noticeable degradation in query

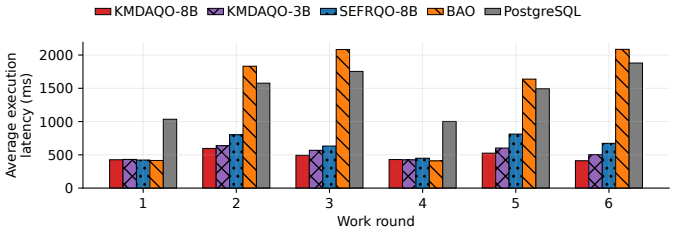

Fig. 10. Performance under Larger Drift Factors on JOB.

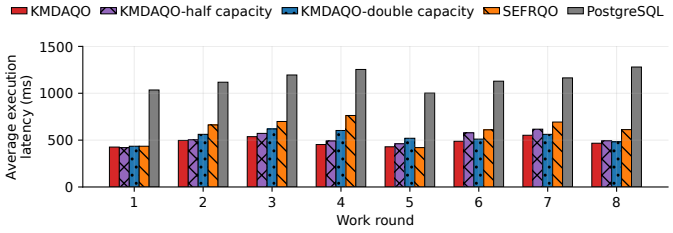

Fig. 11. Performance with Different Knowledge Base Capacity on JOB.

optimization performance. This highlights the importance of knowledge management and editing in preventing the deterioration of optimization quality over time.

On the other hand, when drift awareness is removed, the optimizer exhibits poor adaptability during the initial rounds of workload drift. This early performance degradation occurs because the system is unable to recognize and adjust to the drift in the workload distribution. However, in rounds 5-8, as knowledge accumulates from the previous rounds, the optimizer's performance begins to recover. This suggests that while drift awareness is essential for initial adaptability, accumulated knowledge over time can partially mitigate its absence in later rounds.

E. Sensitivity Analysis

In this subsection, we further evaluate the sensitivity of KMDAQO to two important configuration factors: the workload drift intensity and the capacity of the external knowledge base. These two factors directly affect the difficulty of online retrieval and long-term knowledge maintenance. A larger drift factor makes the incoming workload less similar to previously observed queries, while a smaller knowledge-base capacity forces the system to retain, edit, or prune cases more aggressively.

Figure 10 reports the performance of KMDAQO under larger drift factors on JOB. As the drift factor increases, the workload contains stronger shifts in predicate selectivity and query-pattern distribution, making historical optimization experience less directly reusable. Under this setting, the baselines exhibit more noticeable performance degradation, because learned policies or retrieved examples collected from earlier workload regions may no longer match the current queries. In contrast, KMDAQO-8B maintains consistently lower average latency across rounds. This shows that drift-aware case acquisition can quickly introduce new validated cases for shifted query regions, while bounded knowledge management prevents obsolete cases from dominating retrieval.

We also observe that KMDAQO remains stable across repeated rounds under larger drift factors. This is important because an adaptive optimizer should not only recover after a single workload shift, but also preserve useful knowledge when similar drift patterns reappear later. The repeated-round results indicate that KMDAQO can reuse previously acquired optimization experience instead of relearning from scratch whenever the workload revisits an earlier drift region.

Figure 11 evaluates the impact of the knowledge-base capacity limit. A smaller capacity reduces the number of externally retrievable cases and may trigger earlier pruning or knowledge editing, whereas a larger capacity keeps more historical cases in the retrieval memory. Although these settings change the timing of knowledge maintenance, their impact on final latency is limited. Halving or doubling the capacity only causes small fluctuations in average execution delay, suggesting that KMDAQO does not rely on a finely tuned memory size.

F. Case Study

We further examine a representative drifted query from the JOB workload, namely `4a.sql`, to better understand where the performance gain comes from (Figure 12). The original PostgreSQL plan takes (**213 ms**) and suffers from severe cardinality underestimation in multiple intermediate joins. In particular, the join between `movie_info_idx` and `info_type` is estimated at 1,910 rows but actually produces 117,663 rows, while the join between `movie_keyword` and `keyword` is estimated at 182 rows but actually produces 4,317 rows. After these two branches are joined, PostgreSQL still estimates only 2 rows, whereas the actual result size is 1,940 rows. Due to this large estimation error, PostgreSQL chooses a final nested-loop join against `title` and repeatedly probes `title_pkey`, causing the `Index Scan` on `title` to be executed 5,820 times. In this case, KMDAQO (**102 ms**) generates a coordinated hint set that explicitly guides both join order and join methods, including `Leading((k mk) t) (it mi_idx))`, `HashJoin(k mk)`, `HashJoin(it mi_idx)`, `HashJoin(t mk)`, and `HashJoin(t mi_idx)`. These hints encourage the optimizer to avoid the inefficient final nested-loop probe on `title`, place `title` in a more suitable position in the join tree, and prefer hash joins over enlarged intermediate results. Compared with the default PostgreSQL plan, such a coordinated optimization strategy leads to a more efficient execution plan and lower execution latency.

By contrast, SEFRQO (**139 ms**) adopts a more conservative but suboptimal strategy within the same query optimization setting. It applies only partial guidance, such as `NoNestLoop(t)` or `HashJoin(t mi_idx)`, without explicitly constraining the full join order through `Leading(...)`. This strategy can partially reduce the inefficiency of the PostgreSQL plan, since it alleviates the repeated primary-key probing on `title`. However, it does not fully

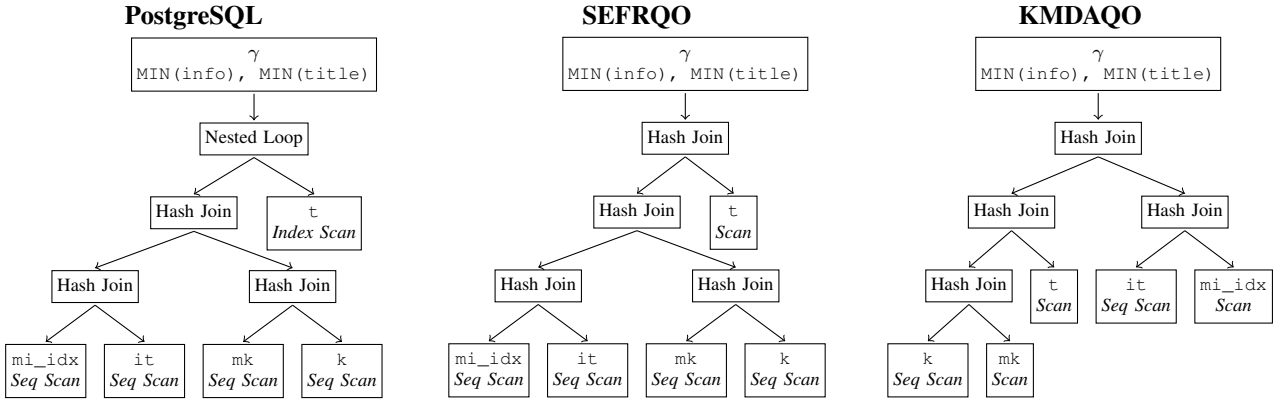

Fig. 12. Simplified query-plan illustration. PostgreSQL places `title` at the final stage and uses a nested-loop probe, SEFRQO improves the final join locally but largely preserves the original join structure, while KMDAQO coordinates both join order and join methods to obtain a more efficient global plan.

address the interaction among cardinality estimation error, join ordering, and intermediate-result growth. As a result, although such local hints may improve upon the default PostgreSQL plan, they are generally less effective than a joint hint set that coordinates both join order and physical operators. This case shows that, for drifted or difficult queries, effective query optimization requires global plan guidance rather than isolated local adjustments. Figure 12 highlights that the main difference is not only the final join operator, but also the global join order: KMDAQO moves `title` earlier in the join tree and avoids the repeated final probe used by PostgreSQL, which further reduces the query latency.

G. End-to-end Time Analysis

The above case study analyzes plan quality from the perspective of database execution time. We further examine the end-to-end cost of KMDAQO under two representative query types: a fast query and a slow analytical query.

KMDAQO does not invoke the LLM for every incoming query. Instead, it adopts a cost-aware serving policy with two paths. For queries whose estimated execution time is short or whose expected optimization benefit is unlikely to amortize the LLM overhead, KMDAQO uses a lightweight fast path. This path only performs routing and optional cache lookup, and then executes the query using PostgreSQL or a previously cached hint. For expensive analytical queries, KMDAQO uses the LLM-assisted path, where the system performs embedding, retrieval, prompt construction, and LLM hint generation before query execution.

For a fast-path query, the end-to-end time is:

$$T_{\mathrm{e2e}}^{\mathrm{fast}} = T_{\mathrm{route}} + T_{\mathrm{DB}},$$

where T_{route} denotes the lightweight routing or cache-checking overhead. This path does not include LLM generation. For a slow-path query, the end-to-end time is:

$$T_{\mathrm{e2e}}^{\mathrm{slow}} = T_{\mathrm{route}} + T_{\mathrm{embed}} + T_{\mathrm{retrieval}} + T_{\mathrm{LLM}} + T_{\mathrm{DB}}.$$

Therefore, the LLM path is used only when the expected reduction in database execution time is large enough to compensate for the optimization overhead.

TABLE II
EXECUTION-TIME AND END-TO-END ANALYSIS FOR FAST-PATH AND SLOW-PATH QUERIES. ALL TIMES ARE REPORTED IN MILLISECONDS.

Query	T_{route}	T_{LLM}	T_{DB}	T_{RAG}	T_{e2e}	T_{PG}
p1	41	0	213	0	254	421
p99	41	4134	7957	62	12194	30000

Table II reports representative results. T_{RAG} in table II refer to the sum of T_{embed} and $T_{\mathrm{retrieval}}$. For the fast query, KMDAQO routes the query to the fast path and does not call the LLM. As a result, its end-to-end time is close to the original database execution time, with only a small routing overhead. This illustrates that KMDAQO avoids imposing the LLM generation cost on short queries, which would otherwise dominate their total latency.

For the slow query, KMDAQO invokes the LLM-assisted path because the query has a much larger optimization opportunity. Although this path introduces LLM inference overhead, the generated hint substantially reduces the database execution time. After adding the online optimization overhead, the end-to-end time remains lower than the baseline execution time. This suggests that KMDAQO can be beneficial for expensive analytical queries, where the execution-time saving can amortize the cost of LLM-based hint generation.

VI. CONCLUSION

In this paper, we propose KMDAQO, a knowledge-managed drift-adaptive query optimization system for LLM-assisted hint generation under evolving workloads. KMDAQO combines retrieval-guided online optimization, drift-aware offline case acquisition, and bounded knowledge maintenance to adapt from execution feedback while limiting stale or redundant retrieval evidence. Experiments on JOB, JOB-EXT, and CEB show that KMDAQO improves average latency, tail robustness, and long-term stability over traditional, learned, and retrieval-augmented baselines.

REFERENCES

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. ACM, 1979, pp. 23–34. [Online]. Available: https://doi.org/10.1145/582095.582099

[2] G. Graefe and W. J. McKenna, "The volcano optimizer generator: Extensibility and efficient search," in *Proceedings of the 9th International Conference on Data Engineering (ICDE)*. Vienna, Austria: IEEE, 1993, pp. 209–218. [Online]. Available: https://dblp.org/rec/conf/icde/GraefeM93

[3] G. Graefe, "The cascades framework for query optimization," *IEEE Data Engineering Bulletin*, vol. 18, no. 3, pp. 19–29, 1995. [Online]. Available: https://dblp.org/rec/journals/debu/Graefe95a

[4] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015. [Online]. Available: https://www.vldb.org/pvldb/vol9/p204-leis.pdf

[5] R. Avnur and J. M. Hellerstein, "Eddies: Continuously adaptive query processing," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Dallas, Texas, USA: ACM, 2000, pp. 261–272. [Online]. Available: https://dblp.org/rec/conf/sigmod/HellersteinA00

[6] M. Stillger, G. M. Lohman, V. Markl, and M. Kandil, "LEO – DB2's LEarning optimizer," in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2001, pp. 19–28. [Online]. Available: https://www.vldb.org/conf/2001/P019.pdf

[7] I. Trummer, "Exact cardinality query optimization with bounded execution cost," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. ACM, 2019, pp. 2–17. [Online]. Available: https://doi.org/10.1145/3299869.3300087

[8] R. Marcus *et al.*, "Bao: Making learned query optimization practical," in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. Virtual Event: ACM, 2021, pp. 1275–1288.

[9] J. Tan, K. Zhao, R. Li, J. X. Yu, C. Piao, H. Cheng, H. Meng, D. Zhao, and Y. Rong, "Can large language models be query optimizer for relational databases?" *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, 2025. [Online]. Available: https://doi.org/10.1145/3769771

[10] Z. Yao, H. Li, J. Zhang, C. Li, and H. Chen, "A query optimization method utilizing large language models," *CoRR*, vol. abs/2503.06902, 2025. [Online]. Available: https://arxiv.org/abs/2503.06902

[11] Z. Zhao, H. Gao, N. Xing, L. Zeng, M. Zhang, G. Chen, M. Rigger, and B. C. Ooi, "Neurbench: Benchmarking learned database components with data and workload drift modeling," *CoRR*, vol. abs/2503.13822, 2025. [Online]. Available: https://arxiv.org/abs/2503.13822

[12] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proceedings of the 2020 NeurIPS*. Vienna, Austria: OpenReview.net, 2020, pp. 9459–9474.

[13] H. Liu, Q. Zhang, R. Marcus, and I. Sabek, "SEFRQO: A self-evolving fine-tuned RAG-based query optimizer," *Proceedings of the ACM on Management of Data*, vol. 3, no. 6, pp. 1–27, Dec. 2025, sIGMOD 2026. [Online]. Available: https://doi.org/10.1145/3769826

[14] ——, "Serag: Self-evolving rag system for query optimization," in *Proceedings of the AI for Data Management Workshop (aiDM) at ACM SIGMOD*. Virtual Event / To Appear: ACM, 2025, pp. 1–12. [Online]. Available: https://viterbi-web.usc.edu/sabek/pdf/25_workshop_serag.pdf

[15] N. Bruno and R. V. Nehme, "Configuration-parametric query optimization for physical design tuning," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. Vancouver, BC, Canada: ACM, 2008, pp. 941–952. [Online]. Available: https://doi.org/10.1145/1376616.1376710

[16] pg_hint_plan Development Group, "pg_hint_plan 1.8 Documentation," https://pg-hint-plan.readthedocs.io/, 2024, accessed: 2026-06-11.

[17] R. Marcus and O. Papaemmanouil, "Deep reinforcement learning for join order enumeration," *CoRR*, vol. abs/1803.00055, 2018. [Online]. Available: https://arxiv.org/abs/1803.00055

[18] R. Marcus *et al.*, "Towards a hands-free query optimizer through deep learning," in *Conference on Innovative Data Systems Research (CIDR)*. Asilomar, CA, USA: CIDR, 2019. [Online]. Available: http://cidrdb.org/cidr2019/papers/p101-marcus-cidr19.pdf

[19] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019. [Online]. Available: https://www.vldb.org/pvldb/vol12/p1705-marcus.pdf

[20] X. Yu, G. Li, C. Chai, and N. Tang, "Reinforcement learning with tree-LSTM for join order selection," in *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1297–1308. [Online]. Available: https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/icde2020-learnedjoinorder.pdf

[21] R. Marcus and O. Papaemmanouil, "Plan-structured deep neural network models for query performance prediction," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1733–1746, 2019. [Online]. Available: https://dblp.org/rec/journals/pvldb/MarcusP19

[22] Z. Yang, W. Chiang, S. Luan, G. Mittal, M. Luo, and I. Stoica, "Balsa: Learning a query optimizer without expert demonstrations," in *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. ACM, 2022, pp. 931–944. [Online]. Available: https://doi.org/10.1145/3514221.3517885

[23] C. Anneser, N. Tatbul, D. Cohen, Z. Xu, P. Pandian, N. Laptev, and R. Marcus, "Autosteer: Learned query optimization for any sql database," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3515–3527, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p3515-anneser.pdf

[24] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou, "Lero: A learning-to-rank query optimizer," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1466–1479, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p1466-zhu.pdf

[25] L. Weng, R. Zhu, D. Wu, B. Ding, B. Zheng, and J. Zhou, "Eraser: Eliminating performance regression on learned query optimizer," *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 926–938, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p926-zhu.pdf

[26] S. Mo, Y. Zhao, Z. Bao, Q. Xu, C. Yang, and G. Cong, "RankPQO: Learning-to-rank for parametric query optimization," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 863–875, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p863-mo.pdf

[27] H. Liu, S. Giridhara, and I. Sabek, "Conformal prediction for verifiable learned query optimization," *Proceedings of the VLDB Endowment*, vol. 18, 2025. [Online]. Available: https://viterbi-web.usc.edu/ sabek/pdf/25_paper_cp4lqo.pdf

[28] X. Yu, C. Chai, G. Li, and J. Liu, "Cost-based or learning-based? a hybrid query optimizer for query plan selection," *Proceedings of the VLDB Endowment*, vol. 15, no. 13, pp. 3924–3936, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p3924-li.pdf

[29] X. Chen, H. Chen, Z. Liang, S. Liu, J. Wang, K. Zeng, H. Su, and K. Zheng, "LEON: A new framework for ML-aided query optimization," *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2261–2273, 2023. [Online]. Available: https://dblp.org/rec/journals/pvldb/ChenCLLWZSZ23

[30] T. Chen, J. Gao, H. Chen, and Y. Tu, "LOGER: A learned optimizer towards generating efficient and robust query execution plans," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1777–1789, 2023. [Online]. Available: https://dblp.org/rec/journals/pvldb/ChenGCT23

[31] L. Woltmann, J. Thiessat, C. Hartmann, D. Habich, and W. Lehner, "FASTgres: Making learned query optimizer hinting effective," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3310–3323, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p3310-habich.pdf

[32] X. Chen, Z. Wang, S. Liu, Y. Li, K. Zeng, B. Ding, J. Zhou, H. Su, and K. Zheng, "BASE: Bridging the gap between cost and latency for query optimization," *Proceedings of the VLDB Endowment*, vol. 16, no. 8, pp. 1958–1966, 2023. [Online]. Available: https://dblp.org/rec/journals/pvldb/ChenWLLZDZSZ23

[33] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper, "Learned cardinalities: Estimating correlated joins with deep learning," *CoRR*, vol. abs/1809.00677, 2018. [Online]. Available: https://arxiv.org/abs/1809.00677

[34] A. Kipf, D. Vorona, J. Müller, T. Kipf, B. Radke, V. Leis, P. Boncz, T. Neumann, and A. Kemper, "Estimating cardinalities with deep sketches," *CoRR*, vol. abs/1904.08223, 2019. [Online]. Available: https://arxiv.org/abs/1904.08223

[35] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, and I. Stoica, "NeuroCard: One cardinality estimator for all tables,"

Proceedings of the VLDB Endowment, vol. 14, no. 1, pp. 61–73, 2021. [Online]. Available: https://vldb.org/pvldb/vol14/p61-yang.pdf

[36] P. Negi, R. Marcus, A. Kipf, H. Mao, N. Tatbul, T. Kraska, and M. Alizadeh, "Flow-loss: Learning cardinality estimates that matter," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2019–2032, 2021. [Online]. Available: https://www.vldb.org/pvldb/vol14/p2019-negi.pdf

[37] J. Liu, W. Dong, Q. Zhou, and D. Li, "Fauce: Fast and accurate deep ensembles with uncertainty for cardinality estimation," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 1950–1963, 2021. [Online]. Available: https://www.vldb.org/pvldb/vol14/p1950-liu.pdf

[38] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou, "Are we ready for learned cardinality estimation?" *Proceedings of the VLDB Endowment*, vol. 14, no. 9, pp. 1640–1654, 2021. [Online]. Available: https://www.vldb.org/pvldb/vol14/p1640-wang.pdf

[39] K. Kim, J. Jung, I. Seo, W. Han, K. Choi, and J. Chong, "Learned cardinality estimation: An in-depth study," in *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. Philadelphia, PA, USA: ACM, 2022, pp. 1214–1227. [Online]. Available: https://doi.org/10.1145/3514221.3526154

[40] B. Hilprecht and C. Binnig, "Zero-shot cost models for out-of-the-box learned cost prediction," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2361–2374, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p2361-hilprecht.pdf

[41] J. Zhang, C. Zhang, G. Li, and C. Chai, "AutoCE: An accurate and efficient model advisor for learned cardinality estimation," in *Proceedings of the 2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2621–2633. [Online]. Available: https://www.computer.org/csdl/proceedings-article/icde/2023/222700c621/1PByR0VqZgI

[42] T. Zeng, J. Lan, J. Ma, W. Wei, R. Zhu, Y. Zhou, P. Li, B. Ding, D. Lian, Z. Wei, and J. Zhou, "PRICE: A pretrained model for cross-database cardinality estimation," *CoRR*, vol. abs/2406.01027, 2024. [Online]. Available: https://arxiv.org/abs/2406.01027

[43] B. Li, Y. Lu, and S. Kandula, "Warper: Efficiently adapting learned cardinality estimators to data and workload drifts," in *Proceedings of the 2022 ACM SIGMOD Conference*. Philadelphia, PA, USA: ACM, 2022, pp. 1150–1163.

[44] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, and M. Alizadeh, "Robust query driven cardinality estimation under changing workloads," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1520–1533, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p1520-negi.pdf

[45] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna, Austria: OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=hSyW5go0v8

[46] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," *Advances in Neural Information Processing Systems*, vol. 35, 2022, rOME. [Online]. Available: https://rome.baulab.info/

[47] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning, "Fast model editing at scale," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://openreview.net/forum?id=0DcZxeWfOPt

[48] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, "Memory-based model editing at scale," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022, pp. 15 817–15 831. [Online]. Available: https://proceedings.mlr.press/v162/mitchell22a.html

[49] P. Wang, Z. Li, N. Zhang, Z. Xu, Y. Yao, Y. Jiang, P. Xie, F. Huang, and H. Chen, "WISE: Rethinking the knowledge memory for lifelong model editing of large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=VJMYOfJVC2

[50] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, W. T. Liang, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui, "Cardinality estimation in DBMS: A comprehensive benchmark evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p752-zhu.pdf